

0	0	1	0	0
1	1	1	1	1
0	1	1	1	1
1	1	1	1	0

Visões das Submatrizes

Patrícia Dockhorn Costa, pdcosta@inf.ufes.br

0	0	1	0	0
1	1	1	1	1
0	1	1	1	1
1	1	1	1	0

Nesta prova, você deve implementar um programa para identificar diferentes visões de submatrizes, dada uma matriz original. Além das funcionalidades básicas de um TAD Matriz (inicialização, impressão e liberação de memória), é pedido duas funcionalidades principais:

1. Achar a visão de uma submatriz específica, dados `linha_ini`, `linha_fim`, `coluna_ini`, `coluna_fim`;
2. Achar visões de todas as submatrizes quadradas de uma dada matriz original.

A terminologia de “visão” está sendo usada pois seu programa **não** deverá alocar memória para os dados finais nas visões, apenas fazer referência (ponteiros) para os dados já existentes na matriz original. No entanto, pode-se alocar memória para a “sentinela” das visões (estrutura inicial e o vetor de ponteiros).

Entrada do Programa

Espera-se o arquivo de entrada (`entrada.txt`) no formato especificado na Figura 1:

```
3 4 linha_coluna
Patricia Joao Maria Joana
Mariana Carlos Beatriz Ana
Pedro Carol Alberto Marilia
1 2 1 3 dados para obter a visão da submatriz, no formato:
        linha_ini linha_fim col_ini col_fim
```

Figura 1 - Formato do arquivo de entrada (`entrada.txt`)

A primeira linha contém 2 números (3 e 4) que representam as dimensões da “matriz” original. Segue-se com as linhas que representam os valores (*strings*) da matriz original; neste exemplo, como a matriz tem 3 linhas, as próximas 3 linhas representam as linhas da matriz original. Após, a última linha do arquivo de entrada especifica os dados para obter a primeira visão da saída, no formato `linha_ini linha_fim col_ini col_fim` (1 2 1 3). Isso significa que a visão deve ser da submatriz das linhas de 1 a 2 e das colunas de 1 a 3, como na Figura 2, em vermelho:

Patricia	Joao	Maria	Joana
Mariana	Carlos	Beatriz	Ana
Pedro	Carol	Alberto	Marilia

Figura 2 - visão da submatriz, dados os parâmetros do arquivo de entrada

Funcionamento do Programa

Sugere-se que seu programa siga os passos:

1. Leia o arquivo de entrada e carregue a matriz original. Imprima na saída;
2. Carregue, também, os parâmetros para a primeira visão;
3. Crie a primeira visão da submatriz, com os parâmetros fornecidos. Imprima na saída. Já pode liberar a memória alocada para esta visão (como é uma visão, a liberação de memória é diferente da liberação de uma matriz comum);
4. Crie (e já imprima na saída) visões de todas as submatrizes quadradas possíveis da matriz original (use a função acima). Vai liberando a memória de cada visão criada;
5. Libere a memória da matriz original.

Saída do Programa

Espera-se, no arquivo de saída (saida.txt), o seguinte: a matriz original, a primeira visão, seguido das visões das submatrizes quadradas. Este enunciado está acompanhado de 3 baterias de testes.

Algoritmos

Seguem versões **simplificadas** dos algoritmos para gerar as visões.

Pseudocódigo para criar visão da submatriz, dados linha_ini linha_fim col_ini col_fim:

```
criaSubMatriz (origem, linha_ini, linha_fim, col_ini, col_fim):
    //definir dimensões da submatriz
    Definir linhas da sub com base nos parâmetros
    Definir colunas da sub com base nos parâmetros

    //não alocar espaço para os valores finais da matriz, como explicado anteriormente
    criar visão sub;

    Para cada índice i de 0 até o número de linhas da submatriz:
        Fazer com que sub->linha[i] aponte para a posição correspondente na matriz
        original, iniciando na linha (lini + i) e coluna col_ini

    retorna sub;
```

Pseudocódigo para criar visões das submatrizes quadradas:

```
imprimeSubVisoesQuadradas (origem):
    Para cada índice i de 0 até o número de linhas da matriz origem
        Para cada índice j de 0 até o número de colunas da matriz origem
            Max_tam: Determinar o maior tamanho possível de submatriz quadrada, a partir de (i,j)
            Para cada x de 0 até o Max_tam:
                sub = criaSubMatriz (origem, i, i+k, j, j+k);
                imprime(sub);
                libera(sub);
```

Regras muito importantes:

- A prova trata de “**visões**”! Não vale alocar espaço para os valores finais nas visões. Deve-se apontar para os valores da matriz original!
- Implementar um TAD Matriz, como discutimos em sala de aula. Ou seja, espero, ao menos, 3 arquivos na resposta (2 para o TAD e um para o cliente – main);
- Usar somente **alocação dinâmica** para todas as matrizes/submatrizes;
- Manipulação de arquivos para entrada somente na main;
- Impressão/saída no arquivo saída.txt pode ser feita no TAD;
- Liberar toda a memória alocada! Use Valgrind...
- MUITO IMPORTANTE: coloque seu nome como primeira linha de todos os arquivos que produzir!
- Deixe a resposta em uma pasta “resposta”.

BOA PROVA! ☺